Máster en Desarrollo Web Full Stack · Conquer Blocks

Proyecto Final de Máster · Memoria

ClinicaCitas

Reservas online y gestión de agenda para clínicas pequeñas

Django · Django REST Framework · React

Vídeo de presentación (5 min): [enlace al vídeo]

Aplicación desplegada: https://clinicacitas.onrender.com

Repositorio: https://github.com/VTrisac/clinicacitas

Usuarios de prueba: paciente · laura · recepcion · admin — contraseña `Clinica2026!`

Alumno	Victor Trisac Pérez
Fecha	Octubre de 2026

Índice

1. Problema
2. Aportación
3. Tecnologías
4. Usuarios y casos de uso
5. Seguridad
6. Arquitectura y modelo de datos
7. Despliegue y pruebas
8. Uso de herramientas de IA
9. Conclusiones
10. Anexo: API REST y usuarios de prueba

1. Problema

Muchas clínicas pequeñas (fisioterapia, psicología, nutrición, dental) siguen dando las citas por teléfono y apuntándolas en una agenda de papel o una hoja de cálculo compartida. Eso tiene tres consecuencias:

- **Recepción vive al teléfono.** Cada cita, cambio o cancelación es una llamada, muchas fuera de horario.
- **Errores en la agenda.** Dos personas apuntan la misma hora, o se da una cita en un día de vacaciones del profesional.
- **Pacientes que no se presentan.** Sin datos, la clínica no sabe cuántas horas pierde ni con qué profesional ocurre más.

Las soluciones comerciales existen, pero suelen ser caras para una clínica de tres o cuatro profesionales y obligan a adaptarse a su forma de trabajar.

2. Aportación

ClinicaCitas centraliza la agenda en una aplicación web con cuatro roles:

- **El paciente reserva solo**, en cuatro pasos, viendo únicamente huecos que existen de verdad: los calcula el servidor a partir del horario del profesional, la duración del servicio, las citas ya reservadas y los bloqueos.
- **La agenda no admite solapes.** La regla se comprueba en el modelo y otra vez en la reserva, dentro de una transacción con bloqueo de fila, para que dos pacientes que pulsan a la vez no se queden con el mismo hueco.
- **Cada rol ve lo que necesita:** el profesional su día, recepción la agenda de todos con alta de citas por teléfono y bloqueos, y administración un panel con la ocupación y la tasa de no presentados.

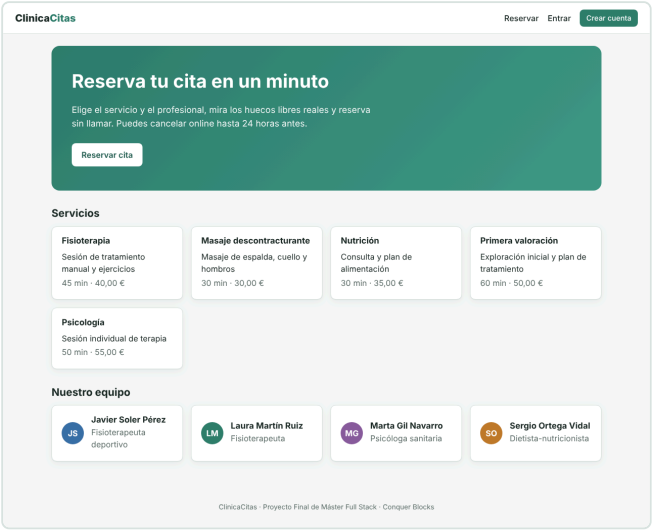


Figura 1. Página de inicio con los servicios y el equipo.

3. Tecnologías

PARTE	TECNOLOGÍA	POR QUÉ
Backend	Python 3.12, Django 5.2	ORM, autenticación, admin y formularios incluidos; es el framework del máster.
API	Django REST Framework 3.18	Serializadores, permisos y vistas genéricas para exponer la agenda a React.
Frontend de reserva	React 19 + Vite 7	El flujo de reserva es interactivo (pasos, calendario semanal, estados de carga) y encaja en una SPA pequeña.
Resto de pantallas	Plantillas de Django y CSS propio	Pantallas internas sencillas; renderizarlas en servidor evita duplicar lógica.
Base de datos	SQLite (local y demo); PostgreSQL con `DATABASE_URL`	El ORM hace que el código sea el mismo con las dos. En PostgreSQL, `SELECT … FOR UPDATE` bloquea la fila en las reservas simultáneas; en SQLite las escrituras ya se serializan.
Despliegue	Render, Gunicorn, WhiteNoise	Blueprint declarativo (`render.yaml`), HTTPS incluido y estáticos servidos por la propia app.
Pruebas	`django.test`, APIClient de DRF, Vitest, Playwright	Tests de lógica, API y vistas; pruebas de extremo a extremo en navegador.

React se compila con Vite dentro de `static/reserva/` con nombres fijos, y la plantilla `reservar.html` lo carga con `{% static %}`. Así React y Django se sirven desde el mismo dominio: no hace falta CORS y la API usa la sesión de Django con su protección CSRF.

4. Usuarios y casos de uso

ROL	CASOS DE USO	PANTALLA
Paciente	Registrarse · reservar cita · ver sus citas · cancelar hasta 24 h antes	/registro/, /reservar/ (React)
Profesional	Ver su agenda por día · marcar atendida o no presentada · guardar notas	/agenda/
Recepción	Ver la agenda de todos · dar cita por teléfono · cancelar · bloquear agenda	/recepcion/
Administración	Ver ocupación y no presentados · gestionar servicios, profesionales y horarios	/panel/, /admin/

Caso principal: el paciente reserva una cita

1. Elige el servicio (por ejemplo, Fisioterapia, 45 min).
2. Elige un profesional que ofrece ese servicio.
3. Ve la semana con los huecos libres y navega entre semanas.
4. Confirma. Si no ha iniciado sesión, se le pide entrar o crear una cuenta.
5. El servidor vuelve a comprobar el hueco; si alguien lo ha cogido entre medias, devuelve un error claro y el paciente elige otro.

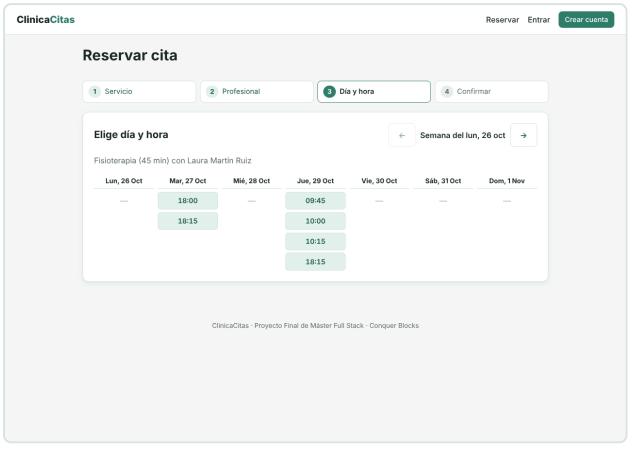


Figura 2. Paso 3: huecos libres de la semana.

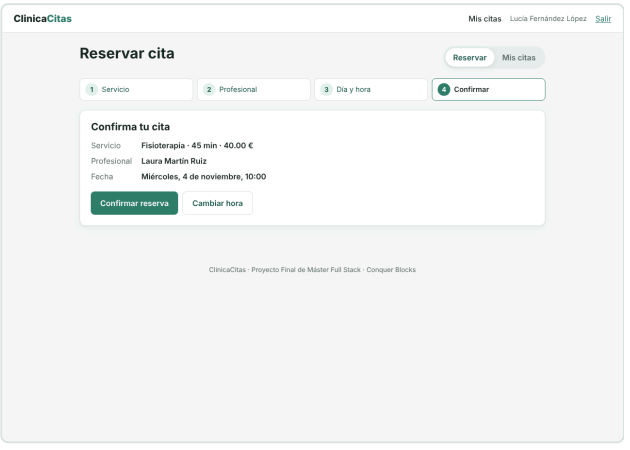


Figura 3. Paso 4: confirmación.

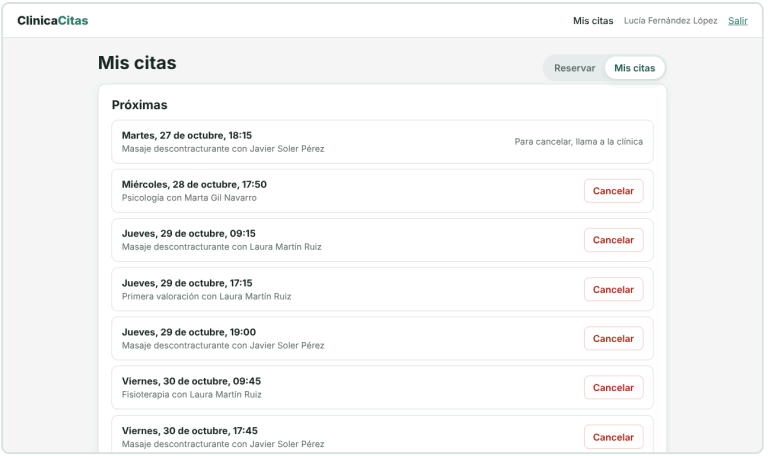


Figura 4. «Mis citas»: próximas (con cancelación) e historial.

Pantallas del personal

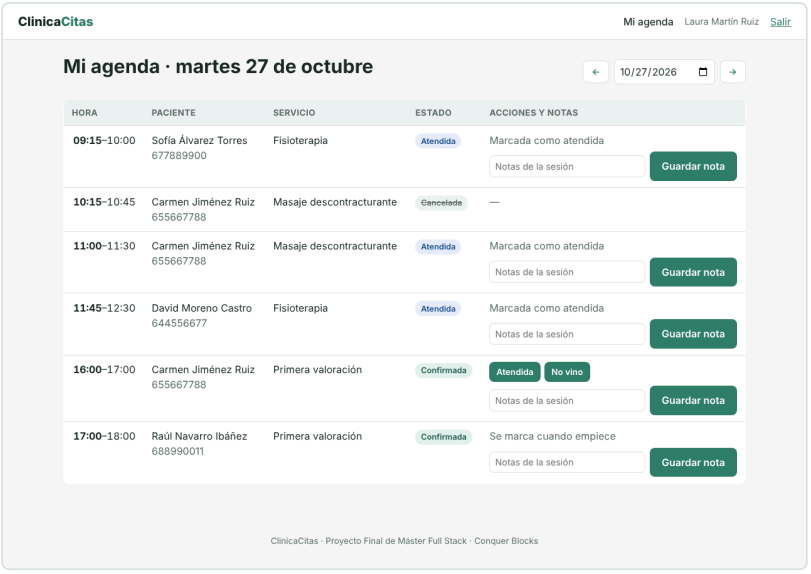


Figura 5. Agenda del profesional: marcar atendida o no presentada y guardar notas.

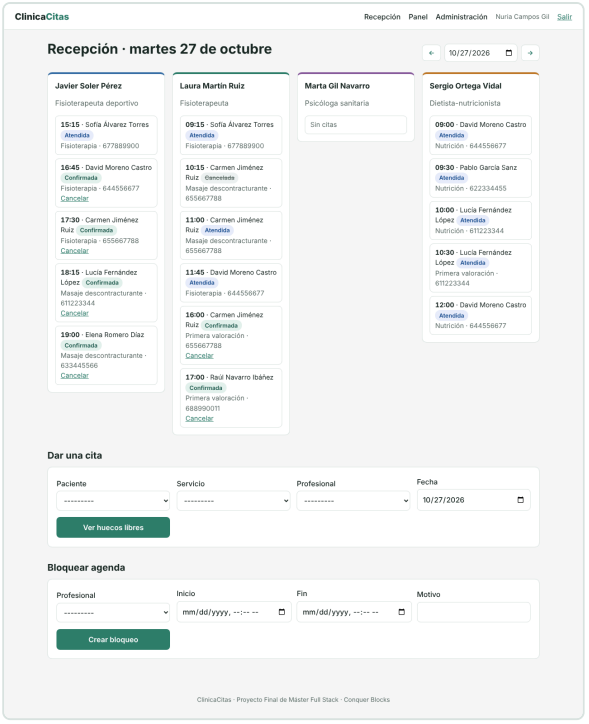


Figura 6. Recepción: agenda de todos los profesionales, alta de citas y bloqueos.

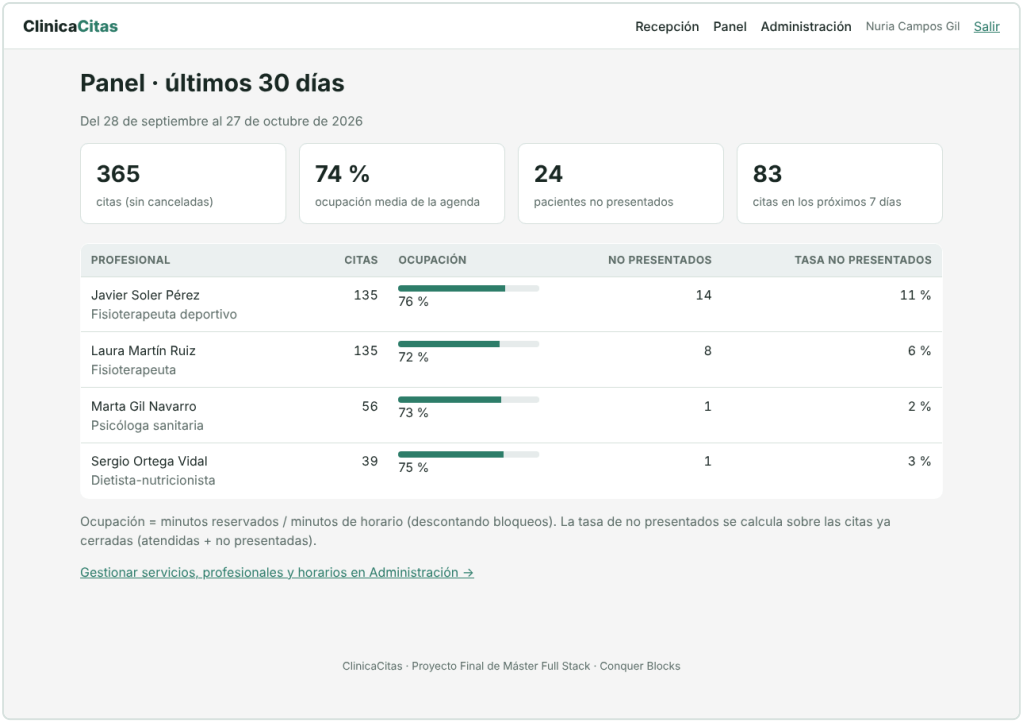


Figura 7. Panel de administración: ocupación y no presentados de los últimos 30 días.

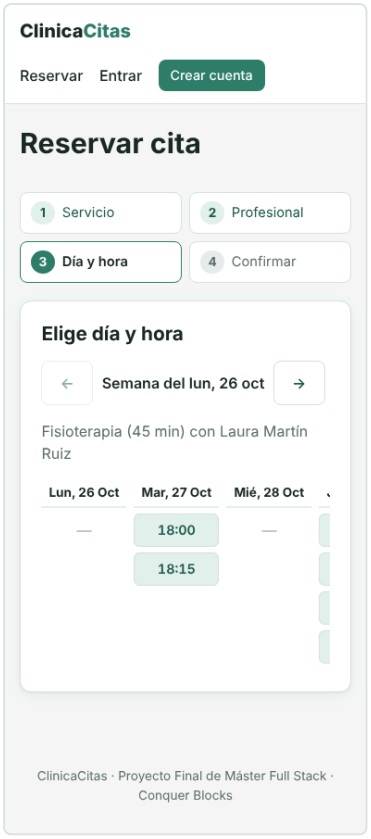


Figura 8. El flujo de reserva en un móvil (390 px).

Reglas de negocio

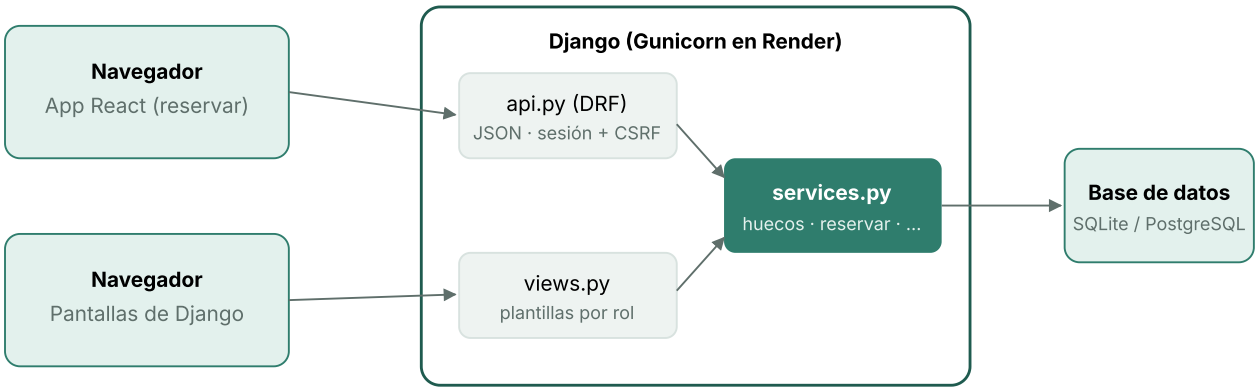
- Los huecos se generan en pasos de 15 minutos y tienen que caber enteros en una franja del horario.
- No se reserva con menos de 1 hora de antelación ni a más de 60 días vista.
- Una cita cancelada libera el hueco.
- Un bloqueo (vacaciones, formación) quita los huecos de ese periodo; si ya había citas, recepción recibe un aviso para llamar a los pacientes.
- Una cita solo se marca como atendida o no presentada cuando ya ha empezado.

Los valores (15 min, 1 h, 24 h, 60 días) están en `settings.CITAS` y se cambian sin tocar el código.

5. Seguridad

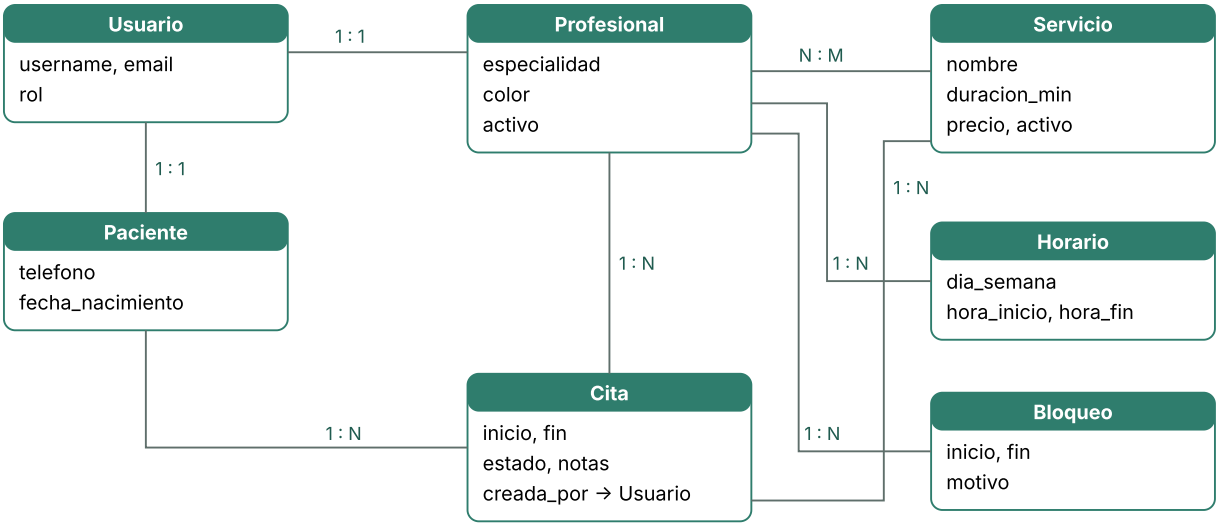
RIESGO	MEDIDA
Acceso a pantallas de otro rol	Decoradores `solo_profesional` y `solo_personal` en las vistas; permiso `EsPaciente` en la API.
Ver o cancelar citas ajenas	La API filtra siempre por el paciente de la sesión (una cita ajena devuelve 404); `services.puede_gestionar()` centraliza quién puede tocar cada cita.
CSRF	Formularios con `{% csrf_token %}`; la API usa `SessionAuthentication` de DRF, que exige el token. React lo lee de la cookie `csrftoken` y lo envía en `X-CSRFToken`.
Reservas simultáneas	`transaction.atomic` + `select_for_update()` sobre el profesional antes de comprobar el hueco.
Datos inválidos	Validación en formularios y serializadores, `clean()` en los modelos y `CheckConstraint` en la base de datos (inicio < fin, duración mínima, precio positivo).
Redirecciones abiertas	El parámetro `volver` se valida con `url_has_allowed_host_and_scheme`.
Contraseñas	Hash de Django (PBKDF2) y validadores de contraseña activados.
Secretos	`SECRET_KEY` y `DATABASE_URL` en variables de entorno; `.env` excluido del repositorio.
Transporte	Con `DEBUG=False`: redirección a HTTPS, cookies `Secure` y HSTS.

6. Arquitectura y modelo de datos



La lógica de negocio vive en `services.py` y no en las vistas: la usan igual la API (React), las pantallas de Django, el comando de datos de ejemplo y los tests. Por eso la regla de «no solapar citas» solo está escrita una vez.

Modelo de datos



Paciente y Profesional son perfiles 1:1 del Usuario, cuyo campo `rol` decide los permisos. Un Profesional ofrece varios Servicios (N:M), tiene franjas de Horario y Bloqueos, y recibe Citas. Cada Cita une Paciente, Profesional y Servicio.

Cálculo de huecos libres

```
para cada franja del horario de ese día de la semana:
    t = inicio de la franja
    mientras t + duración ≤ fin de la franja:
        si t ≥ ahora + 1 h y [t, t + duración) no pisa citas activas ni bloqueos:
            añadir t
        t = t + 15 min
```

7. Despliegue y pruebas

Despliegue

1. El repositorio incluye `render.yaml`, que define el servicio web Python.
2. En Render, *New → Blueprint* crea el servicio. `SECRET_KEY` la genera Render y `DEBUG` va a `False`.
3. `build.sh` instala dependencias, compila React (si hay Node), recopila estáticos con WhiteNoise, aplica migraciones y ejecuta `seed_demo`, que crea servicios, profesionales con horarios, pacientes, unas 600 citas de ejemplo y un usuario por rol.
4. Gunicorn sirve la aplicación por HTTPS.

Decisión sobre la base de datos

La demo usa SQLite, creada y rellenada con los datos de ejemplo en cada despliegue. El plan gratuito de Render solo permite una base PostgreSQL por cuenta y la borra a los 30 días, y la demo tiene que seguir en línea hasta la defensa. La contrapartida es que el disco del plan gratuito no es persistente: las reservas nuevas se pierden cuando Render reinicia el servicio, y la demo vuelve a los datos de ejemplo. En producción bastaría con crear una base PostgreSQL y definir `DATABASE_URL`; el código no cambia.

En el plan gratuito, el servicio se duerme tras 15 minutos sin visitas y la primera petición tarda unos 50 segundos.

Pruebas automáticas

ARCHIVO	TESTS	QUÉ COMPRUEBA
`test_services.py`	21	Huecos (pasos, franjas, citas, cancelaciones, bloqueos, antelación), reservas sin solapes, cancelación por rol y plazo, marcado de estado, estadísticas
`test_api.py`	11	Endpoints públicos, reservar y listar, error 400 con mensaje, aislamiento entre pacientes, CSRF obligatorio, permisos por rol
`test_views.py`	10	Registro, redirección por rol, acceso denegado, recepción da citas y bloquea, agenda, panel, redirección segura
`fechas.test.js`	4	Utilidades de fechas del frontend (Vitest)

Resultado: **42 tests de Django y 4 de Vitest, todos en verde**. Además se ha recorrido el flujo completo en un navegador real con Playwright: reserva anónima hasta el paso de login, inicio de sesión, reserva, «Mis citas», agenda, recepción, panel y vista móvil, sin errores de consola ni desbordamiento horizontal.

```
python manage.py test citas      # backend
cd frontend && npm test           # frontend
```

8. Uso de herramientas de IA

En este proyecto se ha usado un asistente de IA (Claude, de Anthropic) de forma intensiva, y se declara aquí tal como pide la guía del PFM.

FASE	QUÉ HIZO LA IA
Propuesta	Redactar la propuesta de una página a partir de la idea elegida por el alumno.
Backend	Generar los modelos, la lógica de huecos y reservas, la API, las vistas y los formularios.
Frontend	Generar la aplicación React, los estilos y la integración con Django.
Pruebas	Escribir los tests y ejecutar las pruebas en navegador.
Documentación	README, esta memoria y el guion del vídeo.

Cada commit hecho con ayuda de la IA lo indica en el historial con la línea `Co-Authored-By`.

[Completa este apartado con tu parte: qué decisiones tomaste tú, qué revisaste o cambiaste, qué te costó entender y cómo lo comprobaste. Es lo que el tribunal va a preguntar.]

9. Conclusiones

- Se ha construido una aplicación completa con cuatro roles, API REST, una vista React que consume datos reales del backend y despliegue público por HTTPS.
- La decisión técnica más importante fue separar la lógica en `services.py`: una sola implementación de las reglas de la agenda sirve a la web, a la API y a los tests.
- El problema más delicado es la concurrencia en las reservas, resuelto con una transacción y bloqueo de fila además de la validación del modelo.

Mejoras futuras

- Recordatorio por email o SMS el día antes, para reducir los no presentados.
- Lista de espera que avise cuando se libera un hueco.
- Exportar la agenda a CSV o sincronizarla con Google Calendar.
- Pagos o señal al reservar para servicios largos.

10. Anexo: API REST y usuarios de prueba

MÉTODO	RUTA	ACCESO	DESCRIPCIÓN
GET	`/api/servicios/`	Público	Servicios activos
GET	`/api/profesionales/?servicio=`	Público	Profesionales que ofrecen el servicio
GET	`/api/huecos/?profesional=&servicio=&desde=&dias=`	Público	Huecos libres por día
GET	`/api/yo/`	Público	Usuario conectado
GET / POST	`/api/citas/`	Paciente	Mis citas / reservar
POST	`/api/citas/<id>/cancelar/`	Paciente	Cancelar una cita propia

Ejemplo de respuesta de huecos

```
GET /api/huecos/?profesional=1&servicio=1&desde=2026-11-02&dias=2
{
  "dias": [
    {"fecha": "2026-11-02", "huecos": ["2026-11-02T09:00:00+01:00", "2026-11-
02T09:15:00+01:00", ...]},
    {"fecha": "2026-11-03", "huecos": []}
  ],
  "ultimo_dia": "2026-12-26"
}
```

Usuarios de prueba

USUARIO	ROL	QUÉ PROBAR
`paciente`	Paciente	Reservar, ver y cancelar citas
`laura`	Profesional	Agenda del día, marcar estado y notas
`recepcion`	Recepción	Agenda de todos, dar citas y bloquear
`admin`	Administración	Panel de estadísticas y /admin/

Contraseña de todos: `Clinica2026!`